

TEAM B — User Service · Account Service · Portfolio Service

Owner: Mustain Billah Taj (220042166) **Theme:** everything the platform knows about a *person* — who they are, their cash, their shares.

Give this whole file to Claude Code in the repo root and say: "Read TEAM_B_IDENTITY_AND_ASSETS.md and implement all three services completely. *Libs/common/ already exists — read it, import from it, do not rewrite it.*"

You depend on nobody. Your three services never make an outbound call to another team's service. They expose REST and consume RabbitMQ events. You test them alone by publishing fake events with the helper script in §7.

1. THE SHARED CONTRACT (identical in all four team documents — never change unilaterally)

1.1 Your ports and databases

Service	Container	Port	Database
User Service	user-service	8001	postgres-user:5432/user_db (host 5433)
Account Service	account-service	8002	postgres-account:5432/account_db (host 5434)
Portfolio Service	portfolio-service	8006	postgres-portfolio:5432/portfolio_db (host 5438)

Others, for reference: gateway 8000, order 8003, matching-engine 8004, market-data 8005, notification 8007, redis 6379, rabbitmq 5672/15672.

1.2 Money & quantity rules (violating these is the #1 source of wrong numbers)

- Money is `Decimal`, 4 dp, `ROUND_HALF_UP`. **Never float**. Postgres `NUMERIC(18,4)`.
- Money crosses the wire as a **JSON string**: `"195.5000"`. Use `common.money.money_str` in every response model (`field_serializer` or a `Decimal → str` validator).
- Quantity is a positive **int** (whole shares). Postgres `BIGINT`.
- Use `common.money: to_money, money_str, validate_price, validate_quantity, notional`.

1.3 Auth

JWT HS256, secret `JWT_SECRET` identical everywhere. Claims `sub` (user id UUID string), `email`, `username`, `iat`, `exp`, `type:"access"`. Expiry 1440 min. Use `common.security.create_access_token, decode_token, get_current_user`. `/internal/*` endpoints require `X-Internal-Key` — use `Depends(common.security.require_internal_key)`. The gateway refuses to proxy any path containing `/internal/`.

1.4 Event contract

Exchange `exchange.events` (topic, durable), dead-letter `exchange.events.dead`. Envelope:

```
{ "event_id":"<uuid4>", "event_type":"trade.executed",
  "occurred_at":"2026-08-08T12:00:00.000000Z", "version":1, "payload":{ } }
```

Routing key	Published by	You consume it?
order.accepted	Order	no
order.rejected	Order	yes — Account + Portfolio (release reservations)
order.cancel_requested	Order	no
order.cancelled	Matching Engine	yes — Account + Portfolio (release reservations)
order.cancel_rejected	Matching Engine	no
trade.executed	Matching Engine	yes — Account (settle cash) + Portfolio (settle shares)

Payload fields you rely on:

order.rejected	order_id, user_id, symbol, reason, rejected_at
order.cancelled	order_id, user_id, symbol, side, cancelled_quantity, reason, cancelled_at
trade.executed	trade_id, symbol, price(str), quantity(int), buy_order_id, sell_order_id, buyer_user_id, seller_user_id, aggressor_side, buy_order_remaining(int), sell_order_remaining(int), buy_order_limit_price(str null), executed_at

Your queues (declare exactly these names — Team A's RabbitMQ screenshots depend on them):

```
q.account.trade_executed    <- trade.executed
q.account.order_cancelled   <- order.cancelled
```

```
q.account.order_rejected    <- order.rejected
q.portfolio.trade_executed  <- trade.executed
q.portfolio.order_cancelled <- order.cancelled
q.portfolio.order_rejected  <- order.rejected
```

1.5 Redis keys you use

```
lock:funds:{user_id}          distributed lock while touching cash
lock:shares:{user_id}:{SYMBOL} distributed lock while touching a holding
idem:{service}:{event_id}     SET NX EX 86400 – event idempotency
md:last_price:{SYMBOL}       READ ONLY (Market Data writes it) – mark price for unrealized P&L
```

Helpers: `common.redis_client.distributed_lock`, `already_processed`, `clear_processed`, `get_last_price`.

1.6 Internal REST you must provide (Order Service calls these — get them exactly right)

```
POST /internal/reservations (Account)
body {"order_id","user_id","amount":"1020.0000"}
201 {"reservation_id","order_id","amount":"1020.0000","status":"HELD"}
409 {"detail":"insufficient buying power"}
503 {"detail":"could not acquire funds lock"}
Idempotent: same order_id again -> 201 with the EXISTING reservation, no double hold.

POST /internal/reservations/{order_id}/release (Account)
200 {"released":"120.0000"} (0.0000 if nothing left – still 200, never 404 after first release)
404 only if the order_id was never reserved

POST /internal/share-reservations (Portfolio)
body {"order_id","user_id","symbol","quantity":10}
201 {"reservation_id","order_id","quantity":10,"status":"HELD"}
409 {"detail":"insufficient shares"}
Idempotent on order_id.

POST /internal/share-reservations/{order_id}/release (Portfolio)
200 {"released":10}
```

1.7 Every service exposes

`GET /health` → `{"status":"ok","service":"<name>"}`, no DB call. `GET /ready` → checks DB + Redis + broker, 503 if any down. Swagger at `/docs`, `title="<Name> Service"`, `version="1.0.0"`.

1.8 Dockerfile template (build context is the REPO ROOT)

```
FROM python:3.11-slim
WORKDIR /app
ENV PYTHONUNBUFFERED=1 PYTHONDONTWRITEBYTECODE=1 PYTHONPATH=/app:/app/libs
COPY requirements-base.txt ./
COPY services/<NAME>/requirements.txt ./service-requirements.txt
RUN pip install --no-cache-dir -r service-requirements.txt
COPY libs /app/libs
COPY services/<NAME>/app /app/app
COPY services/<NAME>/alembic.ini /app/alembic.ini
COPY services/<NAME>/alembic /app/alembic
COPY services/<NAME>/entrypoint.sh /app/entrypoint.sh
RUN chmod +x /app/entrypoint.sh
EXPOSE <PORT>
CMD ["/app/entrypoint.sh"]
```

`services/<NAME>/requirements.txt` is literally `-r .././requirements-base.txt` plus nothing (you need no extra deps). `entrypoint.sh` — **LF line endings only**:

```
#!/bin/sh
set -e
alembic upgrade head
exec uvicorn app.main:app --host 0.0.0.0 --port "${SERVICE_PORT}"
```

1.9 Alembic (required by the proposal — do it properly)

`alembic/env.py` must convert the async URL to a sync one:

```

from common.config import settings
from common.db import Base, sync_url
from app.models import * # noqa - import so metadata is populated
config.set_main_option("sqlalchemy.url", sync_url(settings.DATABASE_URL))
target_metadata = Base.metadata

```

Hand-write the single initial revision (`0001_initial.py`) — do not rely on `--autogenerate`, the DB may not be up when you generate.

2. SERVICE 1 — USER SERVICE (port 8001, `user_db`)

2.1 Layout

```

services/user-service/
app/{main.py,models.py,schemas.py,routes_auth.py,routes_users.py,deps.py,passwords.py}
alembic/{env.py,versions/0001_initial.py} alembic.ini
requirements.txt Dockerfile entrypoint.sh docker-compose.dev.yml SERVICE_NOTES.md

```

2.2 Model

```

class User(Base):
    __tablename__ = "users"
    id = mapped_column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    email = mapped_column(String(255), nullable=False, unique=True, index=True) # stored LOWERCASE
    username = mapped_column(String(50), nullable=False, unique=True, index=True)
    password_hash = mapped_column(String(255), nullable=False)
    full_name = mapped_column(String(120), nullable=True)
    is_active = mapped_column(Boolean, nullable=False, default=True)
    created_at = mapped_column(DateTime(timezone=True), server_default=func.now())
    updated_at = mapped_column(DateTime(timezone=True), server_default=func.now(), onupdate=func.now())

```

2.3 Passwords — use `bcrypt` directly, **not** passlib

```

import bcrypt
MAX_PASSWORD_BYTES = 72 # bcrypt truncates silently past 72 bytes

def hash_password(raw: str) -> str:
    data = raw.encode("utf-8")
    if len(data) > MAX_PASSWORD_BYTES:
        raise ValueError("password too long (max 72 bytes)")
    return bcrypt.hashpw(data, bcrypt.gensalt(rounds=12)).decode()

def verify_password(raw: str, hashed: str) -> bool:
    try:
        return bcrypt.checkpw(raw.encode("utf-8")[:MAX_PASSWORD_BYTES], hashed.encode())
    except ValueError:
        return False # corrupt hash in DB must not 500

```

2.4 Endpoints

Method	Path	Body / Notes
POST	<code>/auth/register</code>	<code>{email, username, password, full_name?}</code> → 201 <code>{user, access_token, token_type:"bearer", expires_in}</code>
POST	<code>/auth/login</code>	OAuth2 password flow — <code>OAuth2PasswordRequestForm</code> (<code>application/x-www-form-urlencoded</code> , fields <code>username</code> , <code>password</code>). The <code>username</code> field accepts email or username . → 200 <code>{access_token, token_type, expires_in, user}</code>
POST	<code>/auth/login-json</code>	same but JSON <code>{email_or_username, password}</code> — convenience for the frontend
GET	<code>/users/me</code>	JWT → user object
PATCH	<code>/users/me</code>	<code>{full_name?, username?}</code>
POST	<code>/auth/change-password</code>	<code>{current_password, new_password}</code>
GET	<code>/internal/users/{user_id}</code>	X-Internal-Key; returns <code>{id,email,username,full_name}</code> — Notification Service uses it for "mock email" recipients
GET	<code>/internal/users?ids=a,b,c</code>	batch version, max 100 ids

Validation: email via `pydantic.EmailStr`; username `^[a-zA-Z0-9_]{3,30}$`; password 8–72 bytes, must contain a letter and a digit.

Never return `password_hash`. Build a `UserOut` response model and use it everywhere.

2.5 Edge cases — User Service

1. **Duplicate email differing only in case.** `Arham@x.com` vs `arham@x.com`. Lowercase `email` in a pydantic validator *before* the DB write and store lowercase. Username stays case-sensitive for display but uniqueness is checked case-insensitively (`func.lower(User.username)` index).
2. **Concurrent registration race.** Two requests pass the "does it exist" `SELECT` simultaneously. **Do not trust the pre-check** — wrap the `INSERT` in `try/except IntegrityError` → rollback → 409 `{"detail": "email already registered"}`. Add the unique constraint at DB level; that is what actually enforces it.
3. **Password > 72 bytes** (or multi-byte emoji making it >72 bytes). Reject with 400 before hashing. Never let `bcrypt` truncate silently — that is a real auth bypass.
4. **Wrong password vs unknown user must be indistinguishable.** Same 401 body, and run a dummy `bcrypt.checkpw` on a constant hash when the user doesn't exist so the response time doesn't leak account existence.
5. **Inactive user** logging in → 403 `{"detail": "account disabled"}`.
6. **Token for a deleted/disabled user.** `get_current_user` only decodes the JWT — it does not hit the DB. That is intentional (no cross-service DB dependency). In *this* service, endpoints that mutate the user re-load the row and 401 if missing/inactive.
7. **sub must be a string.** `PyJWT` ≥ 2.10 rejects a non-string `sub` on decode. `create_access_token` already does `str(user_id)` — never bypass it.
8. **Login with an email containing uppercase.** Lowercase the identifier before lookup.
9. **Username change collision** → 409, same `IntegrityError` handling as registration.
10. **OAuth2PasswordRequestForm needs python-multipart.** It is in `requirements-base.txt`; if login 500s with "Form data requires python-multipart", that's why.
11. **Empty / whitespace-only full_name** → store `NULL`, not `" "`.
12. **No account row is created here.** The Account Service auto-creates a zero-balance account the first time it sees a user id. This keeps User Service dependency-free — say this in `SERVICE_NOTES.md`.

3. SERVICE 2 — ACCOUNT SERVICE (port 8002, `account_db`)

The most consistency-critical service in the system. The proposal specifically calls out Redis distributed locking and short-lived funds reservations here — this is a graded showpiece, so make the locking visible in the logs.

3.1 Balance model (learn this before writing code)

```
cash_balance    total virtual cash the user owns
held_balance    part of cash_balance reserved against open BUY orders
available_balance = cash_balance - held_balance    <- what buying power actually is
```

DB `CHECK` constraints: `cash_balance >= 0, held_balance >= 0, held_balance <= cash_balance`. If a bug ever tries to break the invariant, Postgres rejects the transaction instead of silently corrupting money.

3.2 Models

```
class Account(Base):
    __tablename__ = "accounts"
    user_id = mapped_column(UUID(as_uuid=True), primary_key=True)
    cash_balance = mapped_column(Numeric(18, 4), nullable=False, default=Decimal("0"))
    held_balance = mapped_column(Numeric(18, 4), nullable=False, default=Decimal("0"))
    currency = mapped_column(String(3), nullable=False, default="USD")
    created_at / updated_at
    __table_args__ = (
        CheckConstraint("cash_balance >= 0", name="ck_cash_non_negative"),
        CheckConstraint("held_balance >= 0", name="ck_held_non_negative"),
        CheckConstraint("held_balance <= cash_balance", name="ck_held_le_cash"),
    )

class Reservation(Base):
    # one per BUY order
    __tablename__ = "reservations"
    order_id = mapped_column(UUID(as_uuid=True), primary_key=True)
    user_id = mapped_column(UUID(as_uuid=True), index=True, nullable=False)
    amount_held = mapped_column(Numeric(18, 4), nullable=False) # original hold
    amount_consumed = mapped_column(Numeric(18, 4), nullable=False, default=0)
    amount_released = mapped_column(Numeric(18, 4), nullable=False, default=0)
    status = mapped_column(String(16), nullable=False, default="HELD") # HELD|CONSUMED|RELEASED
    created_at / updated_at
    # remaining = amount_held - amount_consumed - amount_released

class Transaction(Base):
    __tablename__ = "transactions"
    id = uuid pk
    user_id = UUID index
    type = String(16) # DEPOSIT | HOLD | RELEASE | TRADE_BUY | TRADE_SELL
    amount = Numeric(18,4) # signed: negative reduces cash
    balance_after = Numeric(18,4)
    reference_id = UUID null # order_id or trade_id
    description = String(255)
```

```

created_at    = DateTime index

class ProcessedEvent(Base):
    # belt-and-braces idempotency next to Redis
    __tablename__ = "processed_events"
    event_id     = mapped_column(UUID(as_uuid=True), primary_key=True)
    event_type   = mapped_column(String(64))
    handled_at   = mapped_column(DateTime(timezone=True), server_default=func.now())

```

3.3 Public endpoints

Method	Path	Notes
GET	/account/balance	JWT. Auto-creates the account row on first call. → {user_id, cash_balance, held_balance, available_balance, currency} (all money as strings)
POST	/account/deposit	JWT, {amount:"10000.00"} → new balance + transaction. 0 < amount <= 1000000, ≤4 dp
GET	/account/transactions? limit=50&offset=0&type=	JWT, newest first, limit capped at 200
GET	/internal/accounts/{user_id}	internal key; used by nothing critical, handy for debugging

3.4 Internal endpoints (Order Service calls these synchronously — §1.6 is the contract)

POST /internal/reservations algorithm:

```

async with distributed_lock(redis, f"lock:funds:{user_id}", ttl_ms=5000, wait_seconds=5):
    async with session.begin():
        existing = SELECT reservation WHERE order_id = :order_id FOR UPDATE
        if existing: return 201 with existing          # idempotent replay
        account = SELECT * FROM accounts WHERE user_id = :uid FOR UPDATE # create if missing
        if account.cash_balance - account.held_balance < amount:
            return 409 "insufficient buying power"
        account.held_balance += amount
        insert Reservation(order_id, user_id, amount_held=amount, status="HELD")
        insert Transaction(type="HOLD", amount=-amount, balance_after=available_after, reference_id=order_id)

```

LockTimeout → **503** (never 500, and never proceed without the lock).

POST /internal/reservations/{order_id}/release:

```

lock -> load reservation FOR UPDATE
remaining = amount_held - amount_consumed - amount_released
if remaining <= 0: return {"released":"0.0000"}          # idempotent, still 200
account.held_balance -= remaining                        # clamp at 0 defensively
reservation.amount_released += remaining
reservation.status = "RELEASED" if fully accounted else "HELD"
insert Transaction(type="RELEASE", amount=remaining, reference_id=order_id)

```

POST /internal/reservations/{order_id}/settle — same function the `trade.executed` handler calls, exposed for admin/debug. Body {filled_quantity, price, event_id}. Idempotent on event_id.

3.5 Event handlers

`trade.executed` — one event settles **both** sides. Do it in one transaction per side, buyer first:

```

if await already_processed(redis, "account", event_id): ack and return
notional = to_money(price) * quantity

# --- BUYER ---
lock(lock:funds:{buyer_user_id}):
    reservation = SELECT ... WHERE order_id = buy_order_id FOR UPDATE
    if reservation is None:
        # order placed before this service was up, or MARKET fallback
        # fail-safe: debit directly if funds allow, else log ERROR and record a NEGATIVE-BALANCE incident row
        consume = min(notional, account.cash_balance)
    else:
        remaining_hold = amount_held - amount_consumed - amount_released
        consume = min(notional, remaining_hold)
        reservation.amount_consumed += consume
    account.cash_balance -= notional          # cash actually leaves
    account.held_balance -= consume          # hold is discharged
    Transaction(type="TRADE_BUY", amount=-notional, reference_id=trade_id)
    # price improvement + end of order -> give back what was over-reserved
    if buy_order_remaining == 0 and reservation:

```

```

        release_remaining(reservation)          # same code path as /release

# --- SELLER ---
lock(lock:funds:{seller_user_id}):
    account.cash_balance += notional
    Transaction(type="TRADE_SELL", amount=+notional, reference_id=trade_id)

insert ProcessedEvent(event_id)

```

If anything raises, call `clear_processed(redis,"account",event_id)` before re-raising so the broker retry can actually re-run the handler. **Do this in every handler in every service.**

`order.cancelled` and `order.rejected` → release the remaining reservation for `order_id` (idempotent, no-op if none).

3.6 Edge cases — Account Service

- Double-spend across two browser tabs.** Two `POST /internal/reservations` for the same user land at once. The Redis lock serialises them; the second sees the reduced available balance and 409s. **Log every acquire/release with the user id** — this is your demo evidence for the "distributed locking" requirement.
- Redis down.** `distributed_lock` raises → return **503. Fail closed.** Never reserve funds without the lock; an unreserved order is money created out of nothing.
- Lock expiry mid-transaction.** TTL is 5 s; a slow DB could outlive it. That is why the DB row lock (`SELECT ... FOR UPDATE`) and the `CHECK` constraints exist: the lock is for throughput, the DB is for correctness.
- Releasing a lock you no longer own.** Handled by the Lua compare-and-delete in `common.redis_client` — never `DEL` the key directly.
- Reservation replay.** Order Service retries the POST after a timeout even though the first one succeeded. Idempotent on `order_id` → returns the same reservation, no second hold.
- Release called twice** (once by `order.cancelled`, once by `order.rejected`) → second one releases `0.0000` and returns 200.
- Over-release.** `remaining` is computed, never assumed. Clamp `held_balance` at 0 with `max(Decimal(0), ...)` and let the `CHECK` catch a genuine bug.
- Price improvement.** Buyer reserved at limit 102 but filled at 100. `consume = notional(100)`, and the extra 2/share stays held until the order finishes, then is released. Test this explicitly — it is the most commonly wrong path.
- MARKET buy over-reservation.** Order Service reserves `qty × last_price × 1.05`. After the last fill, `buy_order_remaining == 0` triggers the release of everything unused.
- Partial fills.** One order produces several `trade.executed` events. Each consumes part of the hold. Only the final one (`buy_order_remaining == 0`) releases the remainder.
- Seller has no reservation** — correct by design (sellers reserve *shares*, in Portfolio). Credit cash only; do not look for a funds reservation on the sell side.
- Self-trade** (`buyer_user_id == seller_user_id`). The Matching Engine prevents it, but be defensive: if they are equal, process both legs sequentially in the *same* lock (re-entrant deadlock otherwise — acquire once, do both legs).
- Duplicate trade.executed** after a broker redelivery → `already_processed` + `ProcessedEvent` PK. Two layers, because Redis can be flushed.
- Deposit of "0", "-100", "abc", 1e9, "10.00001"** → all 400 with a clear message. Use `common.money.to_money` and explicit range checks.
- Deposit and order placement concurrently.** Both take `lock:funds:{user_id}` and `SELECT FOR UPDATE`, so they serialise.
- Float contamination.** If any endpoint returns `10000.0` instead of `"10000.0000"`, the frontend will eventually show `10000.000000000002`. Serialise every `Decimal` with `money_str`.
- Account auto-creation race.** Two concurrent first-touches insert the same PK → catch `IntegrityError`, rollback, re-select.
- Events arriving before the account exists** (trade for a user who never called `/balance`). `get_or_create_account` inside the handler.
- ProcessedEvent table growth.** Fine for a demo; note in `SERVICE_NOTES.md` that production would prune rows older than 7 days.
- Transaction listing** must be paginated and indexed on (`user_id, created_at DESC`); a busy bot user generates thousands of rows.

4. SERVICE 3 — PORTFOLIO SERVICE (port 8006, `portfolio_db`)

The proposal's CQRS read-model: holdings and P&L are a **projection of executed trades**, eventually consistent. It also owns **share reservations** for SELL orders (the symmetric counterpart to Account's funds reservations — without it, users can sell shares they don't own).

4.1 Models

```

class Holding(Base):
    __tablename__ = "holdings"
    id = uuid pk
    user_id = UUID, index
    symbol = String(10)
    quantity = BigInteger default 0 CHECK >= 0
    reserved_quantity = BigInteger default 0 CHECK >= 0 and <= quantity
    avg_cost = Numeric(18,4) default 0
    realized_pnl = Numeric(18,4) default 0
    updated_at = DateTime
    __table_args__ = (UniqueConstraint("user_id", "symbol", name="uq_user_symbol"), ...)

class ShareReservation(Base):
    __tablename__ = "share_reservations"
    order_id = UUID pk
    user_id = UUID index
    symbol = String(10)
    quantity = BigInteger # original hold
    consumed = BigInteger default 0
    released = BigInteger default 0
    status = String(16) default "HELD"

```

```

    created_at / updated_at

class TradeHistory(Base):
    __tablename__ = "trade_history"
    id = uuid pk; trade_id = UUID; user_id = UUID index; symbol; side("BUY"/"SELL")
    quantity BigInteger; price Numeric(18,4); realized_pnl Numeric(18,4) default 0; executed_at
    __table_args__ = (UniqueConstraint("trade_id", "user_id", name="uq_trade_user"),)

class ProcessedEvent(Base): # same as Account

```

4.2 Accounting rules (get these exactly right — the grader will check the numbers)

BUY fill — weighted average cost:

```

new_qty = qty + fill_qty
new_avg = to_money((qty * avg_cost + fill_qty * fill_price) / new_qty)
qty, avg_cost = new_qty, new_avg          # realized_pnl unchanged

```

SELL fill — realize against average cost, average cost does **not** change:

```

realized += to_money((fill_price - avg_cost) * fill_qty)
qty      -= fill_qty
reserved_quantity -= min(reserved_quantity, fill_qty)
if qty == 0: avg_cost = 0

```

Unrealized P&L (read time, per holding):

```

mark = redis GET md:last_price:{SYMBOL}  -> if missing: mark = avg_cost, price_stale = true
market_value = mark * quantity
unrealized   = (mark - avg_cost) * quantity

```

4.3 Endpoints

Method	Path	Notes
GET	/portfolio	JWT → {user_id, holdings: [{symbol,name,quantity,available_quantity,reserved_quantity,avg_cost,last_price,market_value,unrealized_pnl,realized_pnl}, ...] } totals:{...}}
GET	/portfolio/pnl	JWT → {total_market_value,total_cost_basis,total_unrealized_pnl,total_realized_pnl,as_of}
GET	/portfolio/trades? limit=50&offset=0&symbol=	JWT, newest first
GET	/portfolio/holdings/{symbol}	JWT, single position; 404 if never held
POST	/internal/share-reservations	internal key, idempotent on order_id
POST	/internal/share-reservations/{order_id}/release	internal key, idempotent
GET	/internal/holdings/{user_id}	internal key, debugging

`available_quantity = quantity - reserved_quantity` — this is what a SELL order can use.

4.4 Event handlers

`trade.executed` — buyer leg and seller leg, both in one idempotency scope:

```

if already_processed(redis,"portfolio",event_id): return
lock lock:shares:{buyer_user_id}:{symbol}: apply BUY fill to buyer's holding (create if absent)
lock lock:shares:{seller_user_id}:{symbol}: apply SELL fill to seller's holding,
                                             consume the share reservation for sell_order_id,
                                             release remainder when sell_order_remaining == 0

insert TradeHistory rows for BOTH users
insert ProcessedEvent(event_id)

```

`order.cancelled` / `order.rejected` → release the share reservation for `order_id` if one exists (BUY orders have none — no-op, 200).

4.5 Edge cases — Portfolio Service

1. **Selling shares you don't own.** POST `/internal/share-reservations` must check `quantity - reserved_quantity >= requested` → else **409 insufficient shares**. Without this the whole exchange can be shorted infinitely.
2. **Selling shares you already have an open sell order for.** `reserved_quantity` is what prevents double-selling the same 10 shares in two orders.
3. **Reservation replay** (Order Service retry) → idempotent on `order_id`.
4. **Release twice** → releases 0, returns 200.
5. **Trade arrives before the buyer has any holding row** → `get_or_create` with qty 0, avg 0, then apply.
6. **Position going to zero.** Keep the row (history + realized P&L live there), set `avg_cost = 0, quantity = 0`. Do **not** delete.
7. **Avg cost rounding drift.** Quantize after every recompute. Over many fills, unquantized division accumulates trailing garbage.
8. **Division by zero** when `new_qty == 0` in the BUY branch — impossible (`fill_qty > 0`), but guard anyway.
9. **Missing mark price** (Redis flushed, or a symbol has never traded) → `mark = avg_cost, unrealized = 0, price_stale: true`. Never crash, never show `null` in the UI.
10. `md:last_price` is a string. `Decimal(redis_value)`, never `float(...)`.
11. **Duplicate trade event** → Redis idem + `uq_trade_user` unique constraint on `TradeHistory`.
12. **Self-trade** (same user both legs) → apply BUY then SELL sequentially; acquire the lock once and do both legs inside it (re-acquiring the same key deadlocks).
13. **Eventual consistency window.** The portfolio may lag the order fill by a few hundred ms. That is the designed CQRS behaviour — document it in `SERVICE_NOTES.md`, and tell Team D to refetch the portfolio ~1 s after a fill notification.
14. **Partial sell fills** consume the reservation incrementally; only the last fill (`sell_order_remaining == 0`) releases the remainder.
15. **Negative quantity** must be impossible — DB `CHECK (quantity >= 0)` and `CHECK (reserved_quantity <= quantity)`. If a handler ever tries, the transaction aborts and the message retries rather than corrupting the book.
16. **A BUY order cancel** hits `/internal/share-reservations/{id}/release` with no reservation → 200, `released: 0`. Do not 404; Order Service releases both sides blindly.
17. **Symbol case.** Always `common.symbols.normalize_symbol` before storing or looking up.
18. **Large portfolios.** GET `/portfolio` reads N holdings and does N Redis GETs — use `redis.mget` for all symbols in one round trip.

5. YOUR `main.py` SKELETON (same shape in all three services)

```
from contextlib import asynccontextmanager
from fastapi import FastAPI
from common.config import settings
from common.db import make_engine, make_sessionmaker, session_dependency
from common.events import Broker, TRADE_EXECUTED, ORDER_CANCELLED, ORDER_REJECTED, Q_ACCOUNT_TRADE_EXECUTED
from common.logging_setup import setup_logging
from common.redis_client import make_redis

setup_logging(settings.SERVICE_NAME, settings.LOG_LEVEL)
engine = make_engine(settings.DATABASE_URL)
Session = make_sessionmaker(engine)
get_session = session_dependency(Session)

@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.redis = make_redis(settings.REDIS_URL)
    app.state.broker = Broker(settings.RABBITMQ_URL, settings.SERVICE_NAME)
    await app.state.broker.connect()
    await app.state.broker.consume(Q_ACCOUNT_TRADE_EXECUTED, [TRADE_EXECUTED], handle_trade_executed)
    await app.state.broker.consume(Q_ACCOUNT_ORDER_CANCELLED, [ORDER_CANCELLED], handle_order_cancelled)
    await app.state.broker.consume(Q_ACCOUNT_ORDER_REJECTED, [ORDER_REJECTED], handle_order_rejected)
    yield
    await app.state.broker.close(); await app.state.redis.aclose(); await engine.dispose()

app = FastAPI(title="Account Service", version="1.0.0", lifespan=lifespan)
```

Consumers run in the same process as the API (aio-pika is async; no separate worker container). Handlers must open their **own** session from `Session()` — never reuse a request-scoped session.

6. LOCAL DEV WITHOUT THE REST OF THE SYSTEM

`services/<name>/docker-compose.dev.yml` — your DB + Redis + RabbitMQ only:

```
services:
  postgres: { image: postgres:16-alpine, environment: {POSTGRES_USER: mse, POSTGRES_PASSWORD: mse_pw, POSTGRES_DB: account_db},
  ports: ["5434:5432"] }
  redis: { image: redis:7-alpine, ports: ["6379:6379"] }
  rabbitmq: { image: rabbitmq:3.13-management-alpine, ports: ["5672:5672", "15672:15672"] }
```

Run the service on the host: `uvicorn app.main:app --reload --port 8002` with `PYTHONPATH=.;..\..\libs` (PowerShell: `$env:PYTHONPATH=".;../../libs"`).

7. TESTING ALONE — fake the Matching Engine

`services/account-service/scripts/emit_event.py` (write it once, copy to portfolio-service):

```
# usage: python emit_event.py trade.executed '{"trade_id": "...", ...}'
import asyncio, json, sys
from common.events import Broker
async def main():
    b = Broker("amqp://guest:guest@localhost:5672/", "tester")
    await b.connect(); await b.publish_event(sys.argv[1], json.loads(sys.argv[2])); await b.close()
asyncio.run(main())
```

Self-check script `services/<name>/selfcheck.py` — plain `asserts`, no `pytest`, must pass before you push:

Account: register-less flow using a hardcoded user UUID — deposit 10000 → balance ok; reserve 2000 → available 8000, held 2000; reserve same order_id again → still held 2000; reserve 9000 → 409; emit `trade.executed` filling 5 @ 200 (limit 210, remaining 5) → cash 9000, held 950 (2100 hold – 1000 consumed – 150 improvement still held); emit final fill remaining 0 → held 0, cash 8000; emit the same event twice → numbers unchanged; release a cancelled order twice → second returns 0.

Portfolio: reserve 10 shares with no holding → 409; apply BUY 10 @ 100 → qty 10 avg 100; BUY 10 @ 200 → qty 20 avg 150; reserve 20 → ok, available 0; SELL 10 @ 180 → qty 10, realized 300, avg still 150; replay the same trade → unchanged; unrealized with `md:last_price:AAPL = 160` → 100.

8. HOUR-BY-HOUR PLAN (24 h)

Hours	Work
0–1	Pull repo, read <code>libs/common</code> end to end, <code>docker compose -f docker-compose.dev.yml up -d</code> .
1–4	User Service complete (models, alembic, register/login/me, bcrypt, all edge cases \$2.5). Swagger green.
4–5	User Service Dockerfile + entrypoint + <code>selfcheck.py</code> . Push branch <code>feat/team-b-identity</code> .
5–11	Account Service complete : models + constraints, deposit, balance, transactions, reservations with Redis lock, event handlers, idempotency.
11–12	Account <code>selfcheck.py</code> passing (the numbers in \$7 must come out exactly). Push .
12–17	Portfolio Service complete : holdings, avg cost / realized / unrealized, share reservations, event handlers.
17–18	Portfolio <code>selfcheck.py</code> passing. Dockerfiles for all three. Push — this is Team A's merge window .
18–21	On call for integration: Team A will hit your <code>/internal/</code> endpoints for real. Fix fast, don't refactor.
21–23	<code>SERVICE_NOTES.md</code> ×3: endpoint list, event handling, edge cases handled, known limitations. Screenshots of Swagger + a lock acquire/release log line.
23–24	Freeze. No new code.

9. DEFINITION OF DONE

- ☐ Three services build from repo root and start clean, `/health` + `/ready` + `/docs` all green.
- ☐ Alembic migration creates every table on an empty DB.
- ☐ All three `selfcheck.py` scripts pass with the exact numbers in \$7.
- ☐ Every money value in every response is a 4-dp **string**.
- ☐ `insufficient buying power` (409) and `insufficient shares` (409) both provably work.
- ☐ Replaying any event twice changes nothing.
- ☐ Redis down → reservations return 503, not 500, and no funds move.
- ☐ Logs show `lock acquired` / `lock released` around every funds mutation.
- ☐ `SERVICE_NOTES.md` written for each service.